

Historically, lip reading existed as a human skill long before it became a computer-vision problem, but modern VSR systems emerged as image-processing and machine-learning methods improved. Earlier systems relied more heavily on hand-crafted visual features and carefully controlled preprocessing, while more recent approaches use deep neural networks to learn visual speech features directly from video. Modern VSR pipelines typically include several major stages, such as visual capture, preprocessing, feature extraction, normalization, and classification. [2] Preprocessing, especially lip-region extraction and frame selection, remains a fundamental part of these systems rather than a minor detail.

In the deep-learning era, VSR systems became significantly more powerful, but also significantly larger. Well-known models such as LipNet [3] helped establish end-to-end visual speech recognition as a practical research direction, while later systems often used deeper visual front ends such as ResNet-based [4] backbones together with temporal sequence modeling. These examples are important because they show the overall direction of the field: modern VSR systems often combine strong spatial feature extraction with temporal modeling, and as a result they are usually designed for GPU-class hardware rather than small microcontrollers.

Rather than attempting a full word or sentence level lip-reading system, I focused on a smaller and more constrained version of the problem: classifying a reduced set of visually meaningful phoneme or viseme groups using heavily preprocessed grayscale mouth images and a lightweight CNN. This narrower formulation was much more realistic for the STM32H7 target, which has limited SRAM, flash, and no GPU support. In that sense, my project fits within the broader history of VSR, but focuses on the feasibility of embedded deployment.

B. External Impact

Visual speech recognition has meaningful potential social impact because it can support speech understanding in situations where audio is unavailable, unreliable, or undesirable. One

important area is accessibility. [2] A system that can infer speech content from mouth movement could support assistive communication tools for people who are hard of hearing, could supplement audio-based speech systems in noisy environments, and could contribute to silent-speech or low-bandwidth communication interfaces. More generally, edge-based VSR could allow some speech-related processing to occur locally [5] without relying entirely on cloud infrastructure, which may be valuable in real-time or low-connectivity settings. For those reasons, VSR has clear relevance to public welfare and human-centered assistive technology.

At the same time, VSR raises serious privacy and fairness concerns because it relies on visual data from human faces. Even when the task is limited to the mouth region, the technology still processes human appearance and behavior in a way that can be sensitive. A system like this could be used responsibly for assistive tools or transcription support, but similar methods could also be misused for surveillance-oriented applications [2] or other forms of non-consensual monitoring. In addition, performance in VSR is closely tied to dataset quality and speaker diversity. Differences in lighting, camera angle, speaking style, skin tone, and facial structure can all affect accuracy, which creates fairness concerns if a system is presented as more universal than it really is. Because of these issues, the limitations of any VSR system must be communicated clearly, and responsible use should emphasize consent, narrow scope, and honest claims about performance.

II. MOTIVATION

A. Tech Contribution

The main technical contribution of this project was reducing the full visual speech recognition problem to a smaller embedded classification task that was still technically meaningful. Instead of attempting full-word or sentence-level lip reading, I focused on whether a lightweight CNN could distinguish a small set of visually meaningful phoneme or viseme groups using heavily preprocessed grayscale mouth images. This made

the problem more realistic for both a one-person capstone project and deployment on an STM32H7 microcontroller.

My contribution was also the full pipeline built around this reduced problem. I developed a preprocessing workflow that started from GRID corpus utterances, isolated the mouth region, aligned phoneme timestamps, split clips by class, and converted the results into grayscale NumPy arrays for CNN training. I then compared lightweight CNNs across different input resolutions, temporal depths, and model sizes, and finally quantized and deployed one of the trained models to the STM32H7. Taken together, the project showed that a simplified VSR pipeline could achieve meaningful performance while remaining small enough for embedded hardware.

B. External Impact

One reason I found this project meaningful is that embedded visual speech recognition could support useful applications without requiring large cloud-based systems. A lightweight on-device model could be helpful in noisy environments, in low-power assistive systems, or as part of a privacy-conscious speech interface where visual data is processed locally instead of being sent to a server. In that sense, reducing the model enough to run on the STM32H7 was not just a technical constraint, but also part of the practical value of the project.

At the same time, the limitations of my implementation also raise important fairness and privacy concerns. The model was trained on a relatively controlled dataset and performed less reliably on my own personal examples, which shows that real-world generalization is still limited. Factors such as lighting, camera angle, speaker appearance, and dataset diversity can all affect performance. Because this is a face-based system, it could also be misused for surveillance or other non-consensual monitoring if applied outside a narrow and transparent scope. For that reason, I think systems like this should be presented honestly: as constrained proof-of-concept tools with potential assistive value, not as universal or deployment-ready lip-reading systems.

III. APPROACH

A. Team Organization

I completed this project as a one-person capstone project. Because of this, I was responsible for all major parts of the system, including dataset acquisition, mouth-region preprocessing, CNN design, model training, quantization, STM32H7 deployment, and final evaluation. In addition to the technical work, I was also responsible for experimental planning, result analysis, figure preparation, and report writing.

B. Plan and Implementation

I originally planned to build a lightweight visual speech recognition pipeline in stages, according to the proposal. The first phase was intended to finalize a small phoneme or viseme set, build the preprocessing pipeline, generate an initial dataset, and train a baseline CNN by Week 3. The second phase was intended to quantize the model, port it to the STM32H7, and document the accuracy versus resource tradeoff by Week 7. After that, the project was meant to expand the phoneme set, perform additional testing, and pursue stretch goals such as improving speed and model size.

In practice, I stayed close to the original proposal, but the timeline shifted significantly. The largest source of delay was preprocessing and dataset construction. Tasks that initially seemed straightforward, such as cropping the mouth region, aligning phoneme timestamps, organizing clips by class, and converting frame windows into compact grayscale arrays, required much more processing and time at larger scales. These couldn't be handled in online python notebooks and instead were shifted to larger on-device scripts. Because these early steps took longer than planned, both the Week 3 offline-training target and the Week 7 embedded-deployment target were missed. A working training pipeline and successful STM32H7 deployment were instead achieved later, around Weeks 9 and 10. Although

the stretch goals were not completed, the main objective of the project was still met: a lightweight visual speech recognition pipeline was trained, quantized, and run successfully on the target embedded hardware. The deployed hardware model was the 32x32x7 Medium network, while the best offline result was later achieved by the 28x28x7 Medium model. Since the deployed model already fit and ran successfully on the STM32H7, the smaller later model would likely also be deployable, although that was not verified experimentally.

C. Theory Acquisition

I first arrived at this project during the brainstorming stage, when I was trying to find a capstone topic that combined speech, image processing, and embedded systems in a way that still felt realistic for a one-person project. I used ChatGPT [6] early in that process as a brainstorming tool to test different project directions, compare scope ideas, and think through whether a simplified visual speech recognition system might be feasible on constrained hardware. That brainstorming helped me narrow the project away from large full-word lip-reading systems and toward a much smaller phoneme/viseme classification problem that could realistically be trained and then deployed on the STM32H7.

My background in ECE 113D and ECE 114 also shaped the project directly. ECE 114 gave me the conceptual foundation for thinking about speech and image signals, especially the importance of preprocessing, resolution tradeoffs, and representing temporal information in a compact form. ECE 113D reinforced the embedded-DSP side of the problem by emphasizing constrained implementation, practical system design, and the need to work within memory and computational limits. Because of those courses, I approached the project less as a general machine-learning problem and more as a signal-processing and embedded-systems tradeoff study. That perspective influenced my decision to use grayscale mouth crops, short temporal windows, and a small CNN instead of a larger end-to-end sequence model.

The most important early technical source I used was the 2023 review *Speaker-independent visual speech recognition: A systematic review and futuristic applications*, which helped me assess whether lightweight VSR was even a reasonable direction to explore. That review frames speaker-independent VSR as a difficult problem because it must interpret high-dimensional mouth motion and cope with variation across speakers, while also surveying recent approaches and future application areas.

This source helped me recognize that I was not trying to compete with full-scale modern VSR systems. Instead, I was trying to test a much smaller and more constrained version of the problem: whether carefully preprocessed visual speech inputs and a compact CNN could still produce meaningful classification performance on embedded hardware.

From there, my theory acquisition focused on the specific ideas I needed to implement the project. I had to build enough background in lip-region preprocessing, [7] phoneme alignment, [1] convolutional neural networks, [8] and post-training quantization [9] to connect the software pipeline to the STM32H7 deployment flow. I learned the preprocessing side largely by working through the practical steps of mouth cropping, transcript/audio alignment, and temporal window generation, while I learned the model side by studying how CNN building blocks such as convolution, batch normalization, ReLU activations, pooling, and softmax classification are used in image-recognition systems. On the deployment side, I also had to learn how INT8 post-training quantization works in TensorFlow Lite and how X-CUBE-AI converts the quantized network into embedded inference code for the target board. In practice, this section of the project was not just literature review; a lot of the theory became clear only after I had to make design choices in preprocessing, training, and embedded validation.

D. Theory

This project used a lightweight 2D convolutional neural network (CNN) to classify short grayscale mouth-image sequences into one

of five phoneme classes. I chose a 2D CNN [8] because the goal of the project was not to build a large full-word visual speech recognition system, but rather to evaluate whether a simplified phoneme classifier could operate under the memory and compute limits of an STM32H7 microcontroller. Instead of using a heavier spatiotemporal architecture such as a 3D CNN [8] or recurrent model, the network accepted a short stack of consecutive grayscale mouth frames as input, allowing temporal information to be represented implicitly through the input depth while keeping the model small enough for embedded deployment.

The CNN consisted of three convolutional stages followed by a compact dense classifier. In each of the first two stages, a 3x3 convolution with padding set to “same” was followed by batch normalization, a ReLU [10] activation, and 2x2 max pooling. The third stage used another 3x3 convolution followed by batch normalization and ReLU, but without an additional pooling step. After feature extraction, global average pooling was applied to reduce each final feature map to a single representative value. A dropout layer with rate 0.30 was then used before a dense hidden layer with 64 units and a final softmax output layer. This structure allowed the network to learn spatial mouth features while keeping the number of trainable parameters relatively small.

Mathematically, each convolution layer applies a learnable filter across the input image to detect local visual patterns such as lip edges, opening shape, and shadow structure. The ReLU activation is given by Figure 2

$$f(x) = \max(0, x)$$

Figure 2 ReLU [10]

which keeps positive activations while setting negative values to zero. This introduces nonlinearity and helps the network learn more expressive visual features. Batch normalization [was applied after each convolution to stabilize intermediate activations and make training more consistent across layers. Max pooling reduced the spatial size of the feature maps, which lowered computational cost and increased the effective receptive field of later layers.

After the convolution blocks, the dense classifier mapped the extracted features into phoneme scores. A dense layer computes each output as Figure 3

$$y_i = \sum_{j=1}^n w_{ij} x_j + b_i$$

Figure 3 Dense Layer

where x_j are the input features, w_{ij} are learned weights, and b_i is a learned bias term. As in figure 4 the final output layer used softmax to convert the class scores into probabilities over the five target classes:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{i=1}^5 e^{z_i}}$$

Figure 4 Softmax

where z_k is the score for class k . The predicted class was taken as the one with the highest softmax probability.

To study the accuracy-resource tradeoff, three model sizes were tested: small, medium, and large [Fig 5]. These models shared the same layer structure but differed in the number of convolution filters. The small model used filter counts of [16, 32, 48], the medium model used [24, 48, 64], and the large model used [32, 64, 96]. Increasing the number of filters gave the network more capacity to learn detailed visual patterns, but also increased model size and computation. This made model width an important design variable for embedded deployment.

Model Size	Conv1 Filters	Conv2 Filters	Conv3 Filters
Small	16	32	48
Medium	24	48	64
Large	32	64	96

Figure 5 Model Sizes

Quantization was also an important part of the embedded workflow. [9] After training, selected

models were converted from floating-point format to INT8 format so they could run efficiently on the STM32H7. In this project, most of the quantization process was handled automatically by the TensorFlow Lite export process and by X-CUBE-AI during embedded code generation. As a result, the main manual requirement during on-board validation was ensuring that the input vectors were quantized into the same INT8 format expected by the deployed model. In general, quantization maps a floating-point value x to an integer value q using a scale ss and zero-point z :

$$q = \text{round}\left(\frac{x}{s}\right) + z$$

Figure 6 Quantization Formula [9]

This reduced memory usage and made embedded inference practical while preserving the overall structure of the trained CNN.

E. Software/Hardware

The project used a mixed software workflow that evolved as the dataset size and preprocessing demands became clearer. I first tried to do the preprocessing in Google Colab, but this proved impractical for the full dataset because execution was too slow and available Google Drive storage was insufficient for the original video corpus. I eventually moved the main preprocessing workflow to a local Python environment in Visual Studio Code using a virtual environment and standalone scripts. This made it possible to run long preprocessing jobs locally over extended periods of time and store the intermediate data outside the limitations of cloud storage. The primary dataset I used was the GRID corpus, [12] using 33 speakers as the basis for training data generation.

Several software tools were used across the preprocessing pipeline. MediaPipe [7] was used for facial landmark detection and mouth-region cropping, while OpenCV was used for frame loading and image handling during preprocessing. Audio and transcript information were extracted from the dataset and used with Montreal Forced Aligner (MFA), [1] run inside a Conda environment, to generate TextGrid alignments containing phoneme timestamps viewable in

Praat. Additional Python scripts were then used to split cropped utterances into phoneme-specific clips, convert frame windows into grayscale NumPy arrays, and resize the resulting inputs into multiple resolutions and frame depths for training. The final dataset variants included combinations such as 48x48x9, 48x48x7, 48x48x5, 32x32x9, 32x32x7, 32x32x5, 28x28x9, 28x28x7, and 28x28x5.

Model development and evaluation were performed in Python using TensorFlow/Keras. After training, selected models were exported as INT8 TensorFlow Lite networks for embedded deployment. The hardware deployment workflow then used X-CUBE-AI within STM32CubeIDE to generate the embedded inference implementation for the STM32H7 microcontroller. I used on-device tests to confirm successful inference and verify that the quantized model fit comfortably within the available flash and RAM limits of the target board. The final deployed configuration used the STM32H7 as the target embedded platform and demonstrated that the lightweight VSR pipeline could run on constrained hardware without GPU support.

F. Operation

The operation of the system can be divided into two parts: building the dataset and model pipeline, and using the trained model for inference. I first processed utterance videos from the GRID corpus to isolate the speaker's mouth region as in Figure 7. This was done by running facial landmark detection and cropping each utterance to a mouth-centered region of interest.



Figure 7 Mouth Isolation with MediaPipe Face Mesh [7]

In parallel, the corresponding audio and text were extracted so that the speech content could later be aligned with the visual data. The aligned transcripts were then processed with Montreal

Forced Aligner to generate TextGrid files containing phoneme-level timestamps viewable in Praat. [Fig 8]

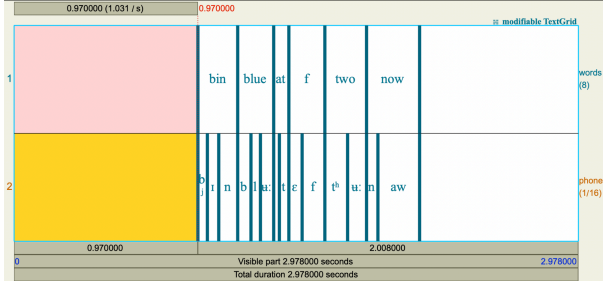


Figure 8 Textgrid file created by Montreal Forced Aligner [1]

These timestamps were used to divide each utterance into phoneme-specific visual clips, which were then converted into grayscale frame windows and saved as NumPy arrays for model training. [Fig 9]

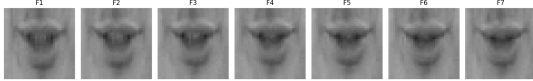


Figure 9 Numpy array frames visualized

Finally, I used additional preprocessing scripts to resize these arrays into multiple image resolutions and temporal depths so that different model configurations could be compared.

Once the dataset had been prepared, the resulting NumPy arrays were used to train lightweight CNN classifiers in Python. Each model took a stack of grayscale mouth images as input and predicted one of the selected phoneme classes.

A key design choice was to evaluate only five output classes: M, SH, O, E, and FV. I chose these classes because they represent distinct mouth shapes while still allowing real visual confusion between similar speech patterns. I also grouped multiple phoneme tokens under each class instead of treating every phoneme label as a separate category. This kept the problem manageable while making it more realistic than a simple one-symbol classifier. Figure 10 shows the grouped phoneme tokens used for the final five-class system.

```
CLASS_NAMES = ["M", "SH", "O", "E", "FV"]
NUM_CLASSES = len(CLASS_NAMES)

# Refined token groups
M_TOKENS = {"b", "b̥", "p", "p̥", "pʰ", "pʲ", "m"}
SH_TOKENS = {"s", "z", "ʃ", "dʒ"}
O_TOKENS = {"ow", "u", "uː"}
E_TOKENS = {"i", "i̥", "iː"}
FV_TOKENS = {"f", "v", "v̥"}
```

Figure 10 Class names and phoneme splits in training

After offline training and evaluation, selected models were quantized to INT8 TensorFlow Lite format and imported into X-CUBE-AI for embedded deployment. The STM32H7 implementation used the generated inference code together with quantized input arrays so that the board could run the classifier directly and return class scores for the five output categories. In this way, the overall system moved from raw video, to cropped and aligned phoneme examples, to offline CNN classification, and finally to on-device embedded inference.

IV. RESULTS

A. Description

I evaluated the system by comparing multiple CNN configurations across image resolution, temporal depth, and model size. Overall, the 28x28 input models performed best on average, followed by the 32x32 models, while the 48x48 models performed worst on average. Across all tested configurations, the average accuracies by image size were 77.7% for 28x28, 76.5% for 32x32, and 72.1% for 48x48. This result suggests that moderate downsampling improved the accuracy-resource tradeoff rather than hurting it, which was encouraging for an embedded deployment target.

The best offline result was achieved by the 28x28x9 Medium model, which reached 82.46% test accuracy on 18,670 examples. Its class-by-class performance was also relatively balanced: M achieved 0.7733 recall, SH achieved 0.8378 recall, O achieved 0.8711 recall, E achieved 0.8331 recall, and FV achieved 0.8104

recall, with a weighted-average F1 score of 0.8255. Figure 11 shows the confusion matrix for this model. Other strong models included the 28x28x5 Large model at 82.42%, the 28x28x9 Large model at 82.42%, and the 28x28x7 Medium model at 81.72%. Taken together, these results showed that the strongest-performing models were concentrated in the 28x28 group rather than at the larger input sizes.

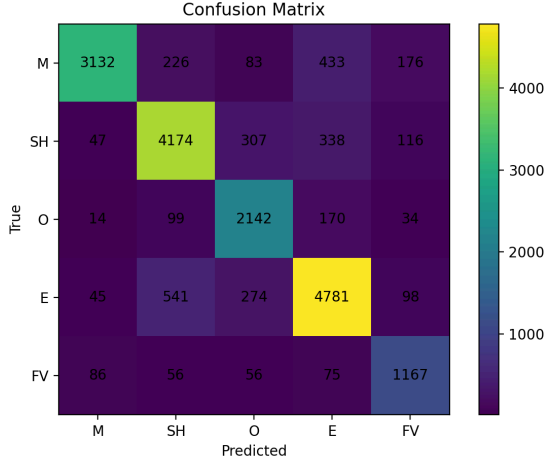


Figure 11 Confusion matrix for 28x28x9 Medium model

The model that I actually deployed to the STM32H7 was the 32x32x7 Medium network, which achieved 80.79% test accuracy offline before deployment. Although it was not the highest-accuracy model in the final sweep, it was one of the strongest validated models available when I was ready to complete the embedded integration. On the board, the quantized model fit comfortably within the available memory budget, requiring about 76 KB of flash and 20 KB of RAM, or only 3.7% of flash and 2% of RAM on the STM32H7 platform. This was one of the most important results of the project, since it showed that a CNN-based visual speech classifier could run on highly constrained hardware while leaving substantial headroom for additional logic or future expansion.

Embedded testing also showed a difference between controlled dataset examples and more informal personal examples. For dataset-based inputs, the deployed model produced strong, highly separated output scores and correctly identified examples such as SH. Personal

examples were less controlled and did not always match the training conditions as closely, but the board was still able to produce correct class predictions in some cases, such as a personal O example shown in the presentation. This result suggests that the system learned meaningful visual structure from the training set, while also showing that real-world generalization was more limited than in the curated test data.

B. Discussion

The overall results suggest that the project succeeded in its main goal: I was able to train a lightweight visual speech classifier that achieved over 80% accuracy in offline testing and also ran successfully on the STM32H7. At the same time, the results show that performance depended strongly on preprocessing quality, input resolution, and how closely the evaluation data matched the training distribution. In particular, the best results were concentrated in the 28x28 models, while larger input sizes did not improve performance as I had initially expected.

One of the most interesting outcomes was that the 28x28 models outperformed the 32x32 and 48x48 models on average. A likely explanation is that moderate downsampling removed some unnecessary visual detail while preserving the larger mouth-shape features that mattered most for this five-class task. Since the phoneme classes in this project were distinguished mainly by broad lip shape, mouth opening, and teeth visibility rather than fine texture, a smaller input resolution may have acted as a useful form of simplification. By contrast, the larger 48x48 inputs likely preserved more variation in lighting, shadow, and speaker-specific appearance, which may have made the classification problem harder rather than easier.

The confusion matrix for the best-performing 28x28x9 Medium model, shown in Fig. 11, also helps explain the system's performance. The strongest class by recall was O, with 2142 correct predictions out of 2459 examples, which suggests that rounded mouth shapes were relatively distinctive for the network. SH and E also performed strongly, with 4174 and 4781 correct predictions respectively. However, the matrix

shows that some classes remained visually ambiguous. The most noticeable confusion was between E and SH, with 541 true E examples predicted as SH, and between M and E, with 433 true M examples predicted as E. This makes sense because several of these classes can share similar mouth appearances depending on speaker, frame timing, and lip transition state. In other words, the network was often not confusing completely unrelated shapes, but rather visually neighboring mouth configurations.

The confusion matrix also shows that performance was not simply limited by one “bad” class. Instead, each class had its own pattern of strengths and weaknesses. M had the highest precision at 0.9422 but lower recall at 0.7733, which means that when the model predicted M it was usually correct, but it still missed a meaningful number of true M examples. FV, on the other hand, had more balanced precision and recall but was harder to judge from raw counts because it had far fewer examples than SH or E. Overall, these class-level results support the idea that the model learned meaningful visual features, but that speaker variation and inter-class visual similarity still limited performance.

The embedded results were especially important because they showed that the project was not only a Python classification exercise. The deployed 32x32x7 Medium model reached 80.79% offline accuracy and required only about 76 KB of flash and 20 KB of RAM on the STM32H7. That memory footprint is very small relative to the available hardware resources, which suggests that the model was well within the practical limits of the board. In my view, this was one of the strongest results of the project, because it demonstrated that a CNN-based VSR model can be made small enough for embedded inference without reducing the problem to something trivial.

At the same time, the difference between dataset-based examples and my own personal examples showed a clear limitation in generalization. On examples drawn from the processed dataset, the board produced strong and confident predictions. On personal examples, performance was more mixed. This gap likely came from differences in lighting, camera angle, speaker appearance, timing, and overall recording

consistency. The training data was relatively controlled, while personal tests introduced variations that the model had not fully learned to handle. This does not mean the model failed; rather, it shows that the current system is best understood as a proof of concept under constrained conditions rather than a robust real-world lip-reading system.

APPENDIX

Code repository:

<https://github.com/equaltojason/embedded-vsr-capstone.git>

REFERENCES

- [1] Montreal Forced Aligner, “Montreal Forced Aligner documentation,” online documentation. [Online]. Available: Montreal Forced Aligner documentation site. Accessed: Mar. 21, 2026.
- [2] P. Nemani, G. S. Krishna, K. Supriya, and S. K. Santosh, “Speaker-independent visual speech recognition: A systematic review and futuristic applications,” *Image and Vision Computing*, vol. 138, Art. no. 104787, 2023.
- [3] Y. M. Assael, B. Shillingford, S. Whiteson, and N. de Freitas, “LipNet: End-to-End Sentence-level Lipreading,” *arXiv preprint arXiv:1611.01599*, 2016.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.
- [5] N. Shrivastava, A. Saxena, Y. Kumar, R. R. Shah, D. Mahata, and A. Stent, “MobiVSR: A Visual Speech Recognition Solution for Mobile Devices,” *arXiv preprint arXiv:1905.03968*, 2019.
- [6] OpenAI, “ChatGPT,” large language model. [Online]. ChatGPT. Accessed: Mar. 21, 2026.
- [7] MediaPipe, “Face Mesh,” online documentation. [Online]. MediaPipe Face Mesh documentation. Accessed: Mar. 21, 2026.

- [8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [9] TensorFlow, “Post-training quantization,” TensorFlow Model Optimization documentation. [Online]. TensorFlow Model Optimization Guide. Accessed: Mar. 21, 2026.
- [10] GeeksforGeeks, “ReLU activation function in deep learning,” online article. [Online]. GeeksforGeeks Deep Learning section. Accessed: Mar. 21, 2026.
- [11] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *Proc. 32nd Int. Conf. on Machine Learning (ICML)*, Lille, France, 2015, pp. 448–456.
- [12] M. Cooke, J. Barker, S. Cunningham, and X. Shao, “An audio-visual corpus for speech perception and automatic speech recognition,” *The Journal of the Acoustical Society of America*, vol. 120, no. 5, pp. 2421–2424, 2006.